🐻Phase 1

1. What is JavaScript?

JavaScript is a programming language that was originally created in **1995 by Brendan Eich** in just **10 days** while he was working at Netscape. It was built to add interactivity to web pages — things like form validation, animations, and pop-upsToday, JavaScript is everywhere:

- **In your browser** — every interactive website uses it (Facebook, YouTube, Gmail).
- **On servers** — through Node.js, you can build backend APIs.
- **In mobile apps** — using frameworks like React Native.
- **In desktop apps** — VS Code itself is built using JavaScript (Electron).
- **In smart devices, games, even robots.**

The "engine" that runs JavaScript in your browser is a piece of software inside the browser. Chrome uses **V8**, Firefox uses **SpiderMonkey**, Safari uses **JavaScriptCore**. Node.js also uses V8 — that's how JS runs outside a browser.

---

2. Setting Up Your Environment

You have **three places** to write and run JavaScript. Use all three at different times.

a) Browser Console (fastest)

1. Open any webpage (Google.com is fine).
2. Press `F12` (or right-click → Inspect).
3. Click the **Console** tab.
4. Type: `2 + 2` and press Enter. You'll see `4`.

This is perfect for trying out small snippets.

b) VS Code + a `.js` file

1. Download **VS Code** (free) from code.visualstudio.com.
2. Create a folder, open it in VS Code, create a file called `app.js`.
3. Write your code inside it.

c) Node.js (to run `.js` files outside the browser)

1. Install **Node.js** from nodejs.org.
2. Open terminal in your folder.
3. Run: `node app.js`

You'll see the output in the terminal.

---

3. Your First Program — `console.log`

`console.log()` prints something to the console. It is the most-used tool in a JS developer's life.

```
console.log("Hello, World!");
console.log(42);
console.log(true);
console.log("My name is", "Aman", "and I am", 25, "years ol
d");
```

You can pass **multiple values** separated by commas, and `console.log` will print them with spaces between.

Other useful console methods:

```
console.log("Normal message");
console.warn("This is a warning");      // shown in yellow in
browsers
console.error("This is an error");      // shown in red
```

```
console.table([1, 2, 3]);                    // prints data as a t
able
```

4. Comments

Comments are notes for humans. JavaScript ignores them.

```
// This is a single-line comment

/*
   This is a
   multi-line comment
*/

console.log("Hello"); // You can also comment at the end of a
line
```

Why comment?

- To remind yourself what code does.
- To temporarily disable code while testing.
- To explain *why* something is done a certain way.

5. Variables — The Heart of Programming

A **variable** is a named box where you store a value. Later, you can use the name to get the value back, or change it.

JavaScript has **three keywords** to declare variables: `var`, `let`, and `const`.

```
var age = 25;
let name = "Aman";
const PI = 3.14159;
```

Declaration vs Initialization

- **Declaration:** creating the variable. → `let x;`
- **Initialization:** giving it a value. → `x = 10;`
- You can combine both. → `let x = 10;`

```
let city;              // declared, value is undefined
console.log(city);     // undefined
city = "Bhopal";       // initialized now
console.log(city);     // Bhopal
```

Basic Differences Between var, let, const

Feature	`var`	`let`	`const`
Can re-assign?	Yes	Yes	**No**
Can re-declare in same scope?	Yes	No	No
Scope	Function	Block	Block
Hoisted?	Yes (as `undefined`)	Yes (TDZ)	Yes (TDZ)

For now, just remember:

- Use `const` by default.
- Use `let` if you know the value will change.
- **Avoid** `var` in modern code (we'll see why in Phase 3 — it has scoping quirks).

```
const country = "India";
let score = 0;
score = score + 10;      // works
// country = "USA";      // ❌ Error: Assignment to constant v
ariable
```

Naming Rules

- Must start with a letter, `_`, or `$`.
- Can contain letters, digits, `_`, `$`.

- **Cannot** start with a digit.
- Cannot use reserved keywords (`let`, `if`, `function`, etc.).
- Case-sensitive: `age` and `Age` are different variables.

Naming Conventions (good style)

- Use **camelCase** for variables: `firstName`, `totalAmount`, `userAge`.
- Use **UPPER_SNAKE_CASE** for true constants: `MAX_USERS`, `PI`.
- Use meaningful names: `let a = 5` is bad. `let studentCount = 5` is good.

---

6. Data Types

Every value in JavaScript has a **type**. JS has two categories:

Primitive Types (7)

Type	Example
`string`	`"Hello"`, `'JS'`, `template`
`number`	`42`, `3.14`, `-7`
`boolean`	`true`, `false`
`null`	`null` (intentional empty value)
`undefined`	`undefined` (no value assigned yet)
`symbol`	`Symbol("id")` (advanced, rarely used early)
`bigint`	`9007199254740993n` (for huge numbers)

Non-Primitive Type

- `object` — covers objects, arrays, functions, dates, etc. We will explore this deeply in Phase 2.

Examples

```
let name = "Aman";          // string
let age = 25;               // number
```

```
let isStudent = true;       // boolean
let car = null;             // null - "no car right now, inten
tionally"
let job;                    // undefined - never assigned
let id = Symbol("uid");     // symbol
let bigNum = 12345678901234567890n; // bigint (note the 'n')
```

null vs undefined — A Common Confusion

- `undefined` means "this variable was never given a value."
- `null` means "I am intentionally setting this to nothing."

```
let a;            // undefined - JS gave it
let b = null;     // null - I gave it
```

---

7. The `typeof` Operator

`typeof` tells you the type of any value.

```
console.log(typeof "hello");      // "string"
console.log(typeof 42);           // "number"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object"  ← famous bug i
n JS!
console.log(typeof {});           // "object"
console.log(typeof []);           // "object"  (arrays are ob
jects)
console.log(typeof function(){}); // "function"
```

Yes, `typeof null` is `"object"`. This is a bug from 1995 that was never fixed because too much code depends on it. Tell your students this story — they'll remember it forever.

---

8. Type Conversion vs Type Coercion

This is one of JavaScript's most famous quirks.

Explicit Conversion (you do it on purpose)

```
let str = "42";
let num = Number(str);      // converts "42" to 42
console.log(typeof num);    // "number"

let n = 100;
let s = String(n);          // converts 100 to "100"

let val = "hello";
let b = Boolean(val);       // true (non-empty string is truth
y)
```

Implicit Coercion (JS does it automatically — often surprisingly)

```
console.log("5" + 3);      // "53"   ← string concatenation
console.log("5" - 3);      // 2      ← number subtraction
console.log("5" * "2");    // 10
console.log(true + 1);     // 2      (true becomes 1)
console.log(false + 1);    // 1      (false becomes 0)
console.log(null + 1);     // 1      (null becomes 0)
console.log(undefined + 1); // NaN  (undefined becomes NaN)
```

The `+` operator is special — if one side is a string, it concatenates. All other math operators try to convert to numbers.

Truthy and Falsy Values

In a boolean context, every value is either *truthy* or *falsy*.

Falsy values (memorize these 6):

```
false, 0, "" (empty string), null, undefined, NaN
```

Everything else is truthy — including `"0"`, `"false"`, `[]`, `{}`.

```
if ("hello")  console.log("truthy");   // runs
if (0)        console.log("won't run");
if ([])       console.log("truthy");   // runs! empty array i
s truthy
```

---

9. Operators

Arithmetic Operators

```
let a = 10, b = 3;

console.log(a + b);   // 13   addition
console.log(a - b);   // 7    subtraction
console.log(a * b);   // 30   multiplication
console.log(a / b);   // 3.333... division
console.log(a % b);   // 1    modulus (remainder)
console.log(a ** b);  // 1000 exponentiation (10^3)
```

Increment and Decrement

```
let x = 5;
x++;   // x is now 6 (post-increment)
++x;   // x is now 7 (pre-increment)
x--;   // x is now 6
--x;   // x is now 5
```

Difference between pre and post:

```
let x = 5;
let y = x++;   // y gets 5 (old value), THEN x becomes 6
let z = ++x;   // x becomes 7 first, THEN z gets 7
```

Assignment Operators

```
let x = 10;
x += 5;   // x = x + 5 → 15
x -= 3;   // x = x - 3 → 12
x *= 2;   // x = x * 2 → 24
x /= 4;   // x = x / 4 → 6
x %= 4;   // x = x % 4 → 2
```

Comparison Operators

```
console.log(5 == "5");    // true   (loose equality — converts types)
console.log(5 === "5");   // false  (strict equality — checks type AND value)
console.log(5 != "5");    // false
console.log(5 !== "5");   // true
console.log(5 > 3);       // true
console.log(5 <= 5);      // true
```

Rule: Always use `===` and `!==`. Avoid `==` and `!=` because they do type coercion and cause hidden bugs.

```
console.log(0 == false);        // true  ← surprising!
console.log("" == false);       // true  ← surprising!
console.log(null == undefined); // true  ← surprising!

console.log(0 === false);       // false ← sane
```

Logical Operators

```
let a = true, b = false;

console.log(a && b);   // false   AND: both must be true
console.log(a || b);   // true    OR: at least one must be tr
ue
console.log(!a);       // false   NOT: flips the value
```

Short-circuit behavior:

- `&&` returns the **first falsy** value, or the last value if all are truthy.
- `||` returns the **first truthy** value, or the last value if all are falsy.

```
console.log("hello" && "world");  // "world"
console.log(0 && "hello");        // 0
console.log("" || "default");     // "default"
console.log("user" || "guest");   // "user"
```

This is useful for setting defaults:

```
let username = userInput || "Guest";
```

Ternary Operator (Shorthand if-else)

```
let age = 20;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status);   // "Adult"
```

Syntax: `condition ? valueIfTrue : valueIfFalse`

---

10. Strings

A string is a sequence of characters wrapped in `' '`, `" "`, or ` `.

```
let s1 = 'Single quotes';
let s2 = "Double quotes";
let s3 = `Backticks (template literals)`;
```

String Concatenation

```
let firstName = "Aman";
let lastName = "Kumar";
let fullName = firstName + " " + lastName;
console.log(fullName);   // "Aman Kumar"
```

Template Literals (modern, preferred)

Use backticks. You can embed variables with `${...}`.

```
let name = "Aman";
let age = 25;

console.log(`Hello, my name is ${name} and I am ${age} years
old.`);
// "Hello, my name is Aman and I am 25 years old."

// Multi-line strings work naturally too:
let poem = `Roses are red,
Violets are blue,
JS is awesome,
And so are you.`;
```

Useful String Methods

```
let s = "Hello, World!";

console.log(s.length);              // 13
console.log(s.toUpperCase());       // "HELLO, WORLD!"
```

```
console.log(s.toLowerCase());        // "hello, world!"
console.log(s.indexOf("World"));     // 7  (position of "Worl
d")
console.log(s.includes("Hello"));    // true
console.log(s.slice(0, 5));          // "Hello"
console.log(s.substring(7, 12));     // "World"
console.log(s.replace("World", "JS"));   // "Hello, JS!"
console.log(s.split(", "));          // ["Hello", "World!"]
console.log("   hi   ".trim());      // "hi"
console.log("abc".repeat(3));        // "abcabcabc"
console.log(s.startsWith("Hello")); // true
console.log(s.endsWith("!"));        // true
console.log(s.charAt(0));            // "H"
console.log(s[0]);                   // "H" (also works)
```

> **Important:** Strings are **immutable**. Methods don't change the original — they return a new string.
>
> ```
> let x = "hello";
> x.toUpperCase();
> console.log(x); // "hello" — unchanged!
> x = x.toUpperCase();
> console.log(x); // "HELLO"
> ```

---

11. Numbers

JavaScript has **one number type** — both integers and decimals are just `number`.

```
let int = 42;
let float = 3.14;
let negative = -100;
let exponent = 5e3;     // 5000
```

Useful Number Methods

```
let n = 3.14159;

console.log(n.toFixed(2));     // "3.14" (returns string!)
console.log(Number("42"));     // 42
console.log(Number("42abc"));  // NaN
console.log(parseInt("42px")); // 42 (parses what it can)
console.log(parseFloat("3.14kg")); // 3.14
console.log(isNaN("hello"));   // true
console.log(Number.isInteger(5));   // true
console.log(Number.isInteger(5.5)); // false
```

The Math Object

`Math` is a built-in object with mathematical methods and constants.

```
console.log(Math.PI);            // 3.14159...
console.log(Math.E);             // 2.71828...

console.log(Math.round(4.6));    // 5
console.log(Math.floor(4.9));    // 4 (always rounds down)
console.log(Math.ceil(4.1));     // 5 (always rounds up)
console.log(Math.abs(-7));       // 7
console.log(Math.max(1, 5, 3));  // 5
console.log(Math.min(1, 5, 3));  // 1
console.log(Math.pow(2, 10));    // 1024
console.log(Math.sqrt(16));      // 4
console.log(Math.random());      // random number between 0 a
nd 1
```

Common pattern — random integer between min and max:

```
let rand = Math.floor(Math.random() * (max - min + 1)) + min;
```

---

12. Conditionals

Conditionals let your program make decisions.

if / else if / else

```
let marks = 75;

if (marks >= 90) {
    console.log("A grade");
} else if (marks >= 75) {
    console.log("B grade");
} else if (marks >= 50) {
    console.log("C grade");
} else {
    console.log("Fail");
}
```

Important: Conditions are evaluated top to bottom. The first matching block runs and the rest are skipped.

Nested if

```
let age = 20;
let hasLicense = true;

if (age >= 18) {
    if (hasLicense) {
        console.log("Can drive");
    } else {
        console.log("Get a license first");
    }
} else {
    console.log("Too young to drive");
}
```

switch Statement

When you have many possible values for one variable, `switch` is often cleaner than long `if-else` chains.

```
let day = "Monday";

switch (day) {
    case "Monday":
        console.log("Start of the week");
        break;
    case "Friday":
        console.log("Weekend coming!");
        break;
    case "Saturday":
    case "Sunday":
        console.log("It's the weekend!");
        break;
    default:
        console.log("Midweek day");
}
```

> **Critical:** Don't forget `break`! Without it, execution "falls through" to the next case. Sometimes that's intentional (like `Saturday` and `Sunday` above), but usually it's a bug.

Note: `switch` uses `===` (strict equality) for comparison.

---

13. Loops

Loops let you repeat code without writing it many times.

The for Loop

The most common loop. Three parts: initialization, condition, update.

```
for (let i = 0; i < 5; i++) {
    console.log("Iteration", i);
```

```
}
// Prints: 0, 1, 2, 3, 4
```

Break down:

- `let i = 0` — runs once at the start
- `i < 5` — checked before each iteration; if true, loop body runs
- `i++` — runs after each iteration

while Loop

Repeats as long as a condition is true.

```
let count = 0;
while (count < 5) {
    console.log(count);
    count++;
}
```

Use `while` when you don't know in advance how many times to loop.

do...while Loop

Like `while`, but **runs at least once** (because the condition is checked at the end).

```
let x = 10;
do {
    console.log(x);
    x++;
} while (x < 5);
// Prints 10 once, even though condition is false
```

for...of Loop (for arrays and strings)

```
let fruits = ["apple", "banana", "mango"];
for (let fruit of fruits) {
```

```
    console.log(fruit);
}

let word = "Hello";
for (let char of word) {
    console.log(char);
}
```

for...in Loop (for objects — brief intro)

```
let person = { name: "Aman", age: 25 };
for (let key in person) {
    console.log(key, ":", person[key]);
}
```

> **We'll cover this in detail in Phase 2 when we study objects.**

---

14. break and continue

break — exit the loop immediately

```
for (let i = 1; i <= 10; i++) {
    if (i === 5) break;
    console.log(i);
}
// Prints: 1, 2, 3, 4
```

continue — skip the current iteration, go to the next

```
for (let i = 1; i <= 5; i++) {
    if (i === 3) continue;
    console.log(i);
```

```
}
// Prints: 1, 2, 4, 5  (3 is skipped)
```

---

15. Taking User Input

In the browser — `prompt()`

```
let name = prompt("What is your name?");
console.log("Hello, " + name);
```

> **Important:** `prompt()` always returns a **string**. If you need a number, convert it:
>
> ```
> let age = Number(prompt("Enter your age:"));
> if (age >= 18) console.log("Adult");
> ```

In Node.js

Use the `readline` module (slightly more complex — we'll cover this later when needed).

---

16. Putting It All Together — Mini Projects

These projects use **only Phase 1 concepts**. Have students attempt all of them.

Project 1: Simple Calculator

```
let num1 = Number(prompt("Enter first number:"));
let operator = prompt("Enter operator (+, -, *, /):");
let num2 = Number(prompt("Enter second number:"));

let result;
if (operator === "+") result = num1 + num2;
else if (operator === "-") result = num1 - num2;
else if (operator === "*") result = num1 * num2;
else if (operator === "/") result = num2 !== 0 ? num1 / num2
```

```
: "Cannot divide by zero";
else result = "Invalid operator";

console.log("Result:", result);
```

Project 2: FizzBuzz (the classic interview question)

Print numbers 1 to 50. But:

- For multiples of 3, print "Fizz"
- For multiples of 5, print "Buzz"
- For multiples of both, print "FizzBuzz"

```
for (let i = 1; i <= 50; i++) {
    if (i % 15 === 0) console.log("FizzBuzz");
    else if (i % 3 === 0) console.log("Fizz");
    else if (i % 5 === 0) console.log("Buzz");
    else console.log(i);
}
```

Project 3: Number Guessing Game

```
let secret = Math.floor(Math.random() * 100) + 1;
let attempts = 0;
let guess;

do {
    guess = Number(prompt("Guess a number between 1 and 10
0:"));
    attempts++;
    if (guess > secret) console.log("Too high!");
    else if (guess < secret) console.log("Too low!");
} while (guess !== secret);
```

```
console.log(`You got it in ${attempts} attempts!`);
```

Project 4: Temperature Converter

Take a temperature and a unit (C or F), convert to the other.

```
let temp = Number(prompt("Enter temperature:"));
let unit = prompt("Is it in C or F?").toUpperCase();

if (unit === "C") {
    console.log(`${temp}°C = ${(temp * 9/5) + 32}°F`);
} else if (unit === "F") {
    console.log(`${temp}°F = ${((temp - 32) * 5/9).toFixed
(2)}°C`);
} else {
    console.log("Invalid unit");
}
```

Project 5: Count Vowels in a String

```
let str = prompt("Enter a string:").toLowerCase();
let vowels = "aeiou";
let count = 0;

for (let char of str) {
    if (vowels.includes(char)) count++;
}

console.log(`Number of vowels: ${count}`);
```

Phase 1 Checklist

By the end of this phase, your students should be able to:

- [ ] Run JavaScript in the browser console, in a `.js` file with Node, and in VS Code.
- [ ] Use `console.log` to debug and print output.
- [ ] Declare variables with `let` and `const` and know when to use each.
- [ ] Name the 7 primitive data types and give an example of each.
- [ ] Recite the 6 falsy values from memory.
- [ ] Explain the difference between `==` and `===`.
- [ ] Write conditional logic using `if/else if/else` and `switch`.
- [ ] Write all four kinds of loops: `for`, `while`, `do-while`, `for...of`.
- [ ] Use `break` and `continue` correctly.
- [ ] Use template literals to build strings cleanly.
- [ ] Use common string and number methods.
- [ ] Take user input and convert it to the correct type.
- [ ] Solve all five mini projects without looking at the solutions.

Common Beginner Mistakes to Watch For

1. **Using `=` instead of `==` or `===` in conditions.** `if (x = 5)` assigns 5 to x and is always truthy. Bug central.
2. **Forgetting that `prompt()` returns a string.** `prompt("Enter age") + 1` → `"251"`, not `26`.
3. **Forgetting `break` in `switch`.**
   Silent fall-through is a classic bug.
4. **Using `var` and getting confused by its scope.**
   Just use `let` and `const`. We'll explore `var`'s weirdness in Phase 3.
5. **Mutating strings expecting them to change.** `str.toUpperCase()` returns a new string — it doesn't change `str`.
6. **Confusing `null` and `undefined`.** `undefined` = JS gave it; `null` = you gave it.
